JavaScript is disabled on your browser.



compact1, compact2, compact3

java.io

# Class Console

java.lang.Object
    java.io.Console

**All Implemented Interfaces:**

Flushable

---

```
public final class Console
extends Object
implements Flushable
```

Methods to access the character-based console device, if any, associated with the current Java virtual machine.

Whether a virtual machine has a console is dependent upon the underlying platform and also upon the manner in which the virtual machine is invoked. If the virtual machine is started from an interactive command line without redirecting the standard input and output streams then its console will exist and will typically be connected to the keyboard and display from which the virtual machine was launched. If the virtual machine is started automatically, for example by a background job scheduler, then it will typically not have a console.

If this virtual machine has a console then it is represented by a unique instance of this class which can be obtained by invoking the `System.console()` method. If no console device is available then an invocation of that method will return `null`.

Read and write operations are synchronized to guarantee the atomic completion of critical operations; therefore invoking methods `readLine()`, `readPassword()`, `format()`, `printf()` as well as the read, format and write operations on the objects returned by `reader()` and `writer()` may block in multithreaded scenarios.

Invoking `close()` on the objects returned by the `reader()` and the `writer()` will not close the underlying stream of those objects.

The console-read methods return `null` when the end of the console input stream is reached, for example by typing control-D on Unix or control-Z on Windows. Subsequent read operations will succeed if additional characters are later entered on the console's input device.

Unless otherwise specified, passing a `null` argument to any method in this class will cause a `NullPointerException` to be thrown.

**Security note:** If an application needs to read a password or other secure data, it should use `readPassword()` or `readPassword(String, Object...)` and manually zero the returned character array after processing to minimize the lifetime of sensitive data in memory.

```
Console cons;
char[] passwd;
if ((cons = System.console()) != null &&
    (passwd = cons.readPassword("[%s]", "Password:")) != null) {
    ...
    java.util.Arrays.fill(passwd, ' ');
}
```

**Since:**

1.6

### *Method Summary*

**Concrete Methods**

| Modifier and Type | Method | Description |
|---|---|---|
| void | **flush**() | Flushes the console and forces any buffered output to be written immediately . |
| **Console** | **format**(**String** fmt, **Object**... args) | Writes a formatted string to this console's output stream using the specified format string and arguments. |
| **Console** | **printf**(**String** format, **Object**... args) | A convenience method to write a formatted string to this console's output stream using the specified format string and arguments. |

| | | |
|---|---|---|
| **Reader** | **reader**() | Retrieves the unique **Reader** object associated with this console. |
| **String** | **readLine**() | Reads a single line of text from the console. |
| **String** | **readLine**(**String** fmt, **Object**... args) | Provides a formatted prompt, then reads a single line of text from the console. |
| char[] | **readPassword**() | Reads a password or passphrase from the console with echoing disabled |
| char[] | **readPassword**(**String** fmt, **Object**... args) | Provides a formatted prompt, then reads a password or passphrase from the console with echoing disabled. |
| **PrintWriter** | **writer**() | Retrieves the unique **PrintWriter** object associated with this console. |

### Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

## *Method Detail*

### writer

```
public PrintWriter writer()
```

Retrieves the unique PrintWriter object associated with this console.

**Returns:**

The printwriter associated with this console

### reader

```
public Reader reader()
```

Retrieves the unique `Reader` object associated with this console.

This method is intended to be used by sophisticated applications, for example, a `Scanner` object which utilizes the rich parsing/scanning functionality provided by the `Scanner`:

```
Console con = System.console();
if (con != null) {
    Scanner sc = new Scanner(con.reader());
    ...
}
```

For simple applications requiring only line-oriented reading, use `readLine(java.lang.String, java.lang.Object...)`.

The bulk read operations `read(char[])` , `read(char[], int, int)` and `read(java.nio.CharBuffer)` on the returned object will not read in characters beyond the line bound for each invocation, even if the destination buffer has space for more characters. The `Reader`'s `read` methods may block if a line bound has not been entered or reached on the console's input device. A line bound is considered to be any one of a line feed (`'\n'`), a carriage return (`'\r'`), a carriage return followed immediately by a linefeed, or an end of stream.

**Returns:**

The reader associated with this console

### format

```
public Console format(String fmt,
                      Object... args)
```

Writes a formatted string to this console's output stream using the specified format string and arguments.

**Parameters:**

`fmt` - A format string as described in `Format string syntax`

`args` - Arguments referenced by the format specifiers in the format string. If there are more arguments than format specifiers, the extra arguments are ignored. The number of arguments is variable and may be zero. The maximum number of arguments is limited by the maximum dimension of a Java array as defined by *The Java™ Virtual*

*Machine Specification*. The behaviour on a null argument depends on the conversion.

**Returns:**

This console

**Throws:**

IllegalFormatException - If a format string contains an illegal syntax, a format specifier that is incompatible with the given arguments, insufficient arguments given the format string, or other illegal conditions. For specification of all possible formatting errors, see the Details section of the formatter class specification.

### printf

```
public Console printf(String format,
                      Object... args)
```

A convenience method to write a formatted string to this console's output stream using the specified format string and arguments.

An invocation of this method of the form `con.printf(format, args)` behaves in exactly the same way as the invocation of

```
con.format(format, args)
```

.

**Parameters:**

format - A format string as described in Format string syntax.

args - Arguments referenced by the format specifiers in the format string. If there are more arguments than format specifiers, the extra arguments are ignored. The number of arguments is variable and may be zero. The maximum number of arguments is limited by the maximum dimension of a Java array as defined by *The Java™ Virtual Machine Specification*. The behaviour on a null argument depends on the conversion.

**Returns:**

This console

**Throws:**

IllegalFormatException - If a format string contains an illegal syntax, a format specifier that is incompatible with the given arguments, insufficient arguments given the format string, or other illegal conditions. For specification of all possible formatting errors, see the Details section of the formatter class specification.

### readLine

```
public String readLine(String fmt,
                       Object... args)
```

Provides a formatted prompt, then reads a single line of text from the console.

**Parameters:**

`fmt` - A format string as described in Format string syntax.

`args` - Arguments referenced by the format specifiers in the format string. If there are more arguments than format specifiers, the extra arguments are ignored. The maximum number of arguments is limited by the maximum dimension of a Java array as defined by *The Java™ Virtual Machine Specification*.

**Returns:**

A string containing the line read from the console, not including any line-termination characters, or null if an end of stream has been reached.

**Throws:**

`IllegalFormatException` - If a format string contains an illegal syntax, a format specifier that is incompatible with the given arguments, insufficient arguments given the format string, or other illegal conditions. For specification of all possible formatting errors, see the Details section of the formatter class specification.

`IOError` - If an I/O error occurs.

### readLine

```
public String readLine()
```

Reads a single line of text from the console.

**Returns:**

A string containing the line read from the console, not including any line-termination characters, or null if an end of stream has been reached.

**Throws:**

`IOError` - If an I/O error occurs.

### readPassword

```
public char[] readPassword(String fmt,
                           Object... args)
```

Provides a formatted prompt, then reads a password or passphrase from the console with echoing disabled.

**Parameters:**

fmt - A format string as described in Format string syntax for the prompt text.

args - Arguments referenced by the format specifiers in the format string. If there are more arguments than format specifiers, the extra arguments are ignored. The maximum number of arguments is limited by the maximum dimension of a Java array as defined by *The Java™ Virtual Machine Specification*.

**Returns:**

A character array containing the password or passphrase read from the console, not including any line-termination characters, or null if an end of stream has been reached.

**Throws:**

IllegalFormatException - If a format string contains an illegal syntax, a format specifier that is incompatible with the given arguments, insufficient arguments given the format string, or other illegal conditions. For specification of all possible formatting errors, see the Details section of the formatter class specification.

IOError - If an I/O error occurs.

### readPassword

```
public char[] readPassword()
```

Reads a password or passphrase from the console with echoing disabled

**Returns:**

A character array containing the password or passphrase read from the console, not including any line-termination characters, or null if an end of stream has been reached.

**Throws:**

IOError - If an I/O error occurs.

### flush

```
public void flush()
```

Flushes the console and forces any buffered output to be written immediately .

**Specified by:**

`flush` in interface `Flushable`



Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.